

▼ Bài 1:

```
from transformers import pipeline
# 1. Tải pipeline "fill-mask"
# Pipeline này sẽ tự động tải một mô hình mặc định phù hợp (thường là một biến thể của BERT)
mask_filler = pipeline("fill-mask")
# 2. Câu đầu vào với token [MASK]
input_sentence = "Hanoi is the (<mask>) of Vietnam."
# 3. Thực hiện dự đoán
# top_k=5 yêu cầu mô hình trả về 5 dự đoán hàng đầu
predictions = mask_filler(input_sentence, top_k=5)
# 4. In kết quả
print(f"Câu gốc: {input_sentence}")
for pred in predictions:
    print(f"Dự đoán: '{pred['token_str']}' với độ tin cậy: {pred['score']:.4f}")
    print(f"-> Câu hoàn chỉnh: {pred['sequence']}")
```

No model was supplied, defaulted to distilbert/distilroberta-base and revision fb53ab8 (<https://huggingface.co/distilbert/distilroberta-base>). Using a pipeline without specifying a model name and revision in production is not recommended.

Some weights of the model checkpoint at distilbert/distilroberta-base were not used when initializing RobertaForMaskedLM: ['roberta.pooler.dense.weight', 'roberta.pooler.dense.bias']. This IS expected if you are initializing RobertaForMaskedLM from the checkpoint of a model trained on another task or with another device set to use cpu. This IS NOT expected if you are initializing RobertaForMaskedLM from the checkpoint of a model that you expect to be exactly identical.

Câu gốc: Hanoi is the (<mask>) of Vietnam.
 Dự đoán: 'capital' với độ tin cậy: 0.4033
 -> Câu hoàn chỉnh: Hanoi is the (capital) of Vietnam.
 Dự đoán: 'Republic' với độ tin cậy: 0.3443
 -> Câu hoàn chỉnh: Hanoi is the (Republic) of Vietnam.
 Dự đoán: 'north' với độ tin cậy: 0.0417
 -> Câu hoàn chỉnh: Hanoi is the (north) of Vietnam.
 Dự đoán: 'state' với độ tin cậy: 0.0312
 -> Câu hoàn chỉnh: Hanoi is the (state) of Vietnam.
 Dự đoán: 'south' với độ tin cậy: 0.0246
 -> Câu hoàn chỉnh: Hanoi is the (south) of Vietnam.

- Mô hình đã dự đoán đúng "capital" với xác suất xuất hiện cao nhất.
- BERT đọc câu theo cả hai chiều (trái và phải cùng lúc). Bên cạnh đó các mô hình như BERT cũng được huấn luyện bằng cách điền từ bị che nên rất phù hợp cho các tác vụ này.

▼ Bài 2:

```
from transformers import pipeline
# 1. Tải pipeline "text-generation"
# Pipeline này sẽ tự động tải một mô hình phù hợp (thường là GPT-2)
generator = pipeline("text-generation")
# 2. Đoạn văn bản mời
prompt = "The best thing about learning NLP is"
# 3. Sinh văn bản
# max_length: tổng độ dài của câu mời và phần được sinh ra
# num_return_sequences: số lượng chuỗi kết quả muốn nhận
generated_texts = generator(prompt, max_length=50, num_return_sequences=1)
# 4. In kết quả
print(f"\nCâu mời: '{prompt}'")
for text in generated_texts:
    print("Văn bản được sinh ra:")
    print(text['generated_text'])
```

No model was supplied, defaulted to openai-community/gpt2 and revision 607a30d (<https://huggingface.co/openai-community/gpt2>). Using a pipeline without specifying a model name and revision in production is not recommended.

Device set to use cpu

Truncation was not explicitly activated but `max\_length` is provided a specific value, please use `truncation=True` to explicitly set the truncation strategy. Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Both `max\_new\_tokens` (=256) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the docs for more details.

Câu mời: 'The best thing about learning NLP is'

Văn bản được sinh ra:

The best thing about learning NLP is that you get to learn it in a classroom.

Why this is a bad idea

The first reason to train your NLP skills is that you want to learn the same thing over and over. You want to learn something new

The second reason to train your NLP skills is that you want to learn it in a group. You want to learn something new, to learn so

But it's not just about learning, it's about having fun, having fun in a group.

NLP is the art of learning. It's how you learn to love, to look at something, and to talk about it. When you do learn something

NLP is not a matter of having fun. NLP is the art of thinking. It's the art of thinking about something. It's the art of taking

When you do your best, you learn to love the world because you love it. Then you learn to love something else.

It's hard, and it takes time – but

- Kết quả sinh ra khá hợp lí nhưng không có domain, dường như không hiểu NLP là gì mà mô hình chỉ tập trung vào từ `learn` để sinh các từ tiếp theo.
- GPT phù hợp vì nó đọc văn bản từ trái sang phải và được huấn luyện đúng tác vụ dự đoán từ tiếp theo.

▼ Bài 3:

```
import torch
from transformers import AutoTokenizer, AutoModel

# 1. Chọn một mô hình BERT
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# 2. Câu đầu vào
sentences = ["This is a sample sentence."]

# 3. Tokenize câu
# padding=True: đệm các câu ngắn hơn để có cùng độ dài
# truncation=True: cắt các câu dài hơn
# return_tensors='pt': trả về kết quả dưới dạng PyTorch tensors
inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors='pt')

# 4. Đưa qua mô hình để lấy hidden states
# torch.no_grad() để không tính toán gradient, tiết kiệm bộ nhớ
with torch.no_grad():
    outputs = model(**inputs)
# outputs.last_hidden_state chứa vector đầu ra của tất cả các token
last_hidden_state = outputs.last_hidden_state
# shape: (batch_size, sequence_length, hidden_size)

# 5. Thực hiện Mean Pooling
# Để tính trung bình chính xác, chúng ta cần bỏ qua các token đệm (padding tokens)
attention_mask = inputs['attention_mask']
mask_expanded = attention_mask.unsqueeze(-1).expand(last_hidden_state.size()).float()
sum_embeddings = torch.sum(last_hidden_state * mask_expanded, 1)
sum_mask = torch.clamp(mask_expanded.sum(1), min=1e-9)
sentence_embedding = sum_embeddings / sum_mask

# 6. In kết quả
print("Vector biểu diễn của câu:")
print(sentence_embedding)
print("\nKích thước của vector:", sentence_embedding.shape)
```

```
Vector biểu diễn của câu:
tensor([[ -6.3874e-02, -4.2837e-01, -6.6779e-02, -3.8430e-01, -6.5784e-02,
         -2.1826e-01,  4.7636e-01,  4.8659e-01,  4.0647e-05, -7.4273e-02,
        -7.4740e-02, -4.7635e-01, -1.9773e-01,  2.4824e-01, -1.2162e-01,
         1.6678e-01,  2.1045e-01, -1.4576e-01,  1.2636e-01,  1.8635e-02,
         2.4640e-01,  5.7090e-01, -4.7014e-01,  1.3782e-01,  7.3650e-01,
        -3.3808e-01, -5.0331e-02, -1.6452e-01, -4.3517e-01, -1.2900e-01,
         1.6516e-01,  3.4004e-01, -1.4930e-01,  2.2422e-02, -1.0488e-01,
        -5.1916e-01,  3.2964e-01, -2.2162e-01, -3.4206e-01,  1.1993e-01,
        -7.0148e-01, -2.3126e-01,  1.1224e-01,  1.2550e-01, -2.5191e-01,
        -4.6374e-01, -2.7261e-02, -2.8415e-01, -9.9249e-02, -3.7017e-02,
        -8.9192e-01,  2.5005e-01,  1.5816e-01,  2.2701e-01, -2.8497e-01,
         4.5300e-01,  5.0945e-03, -7.9441e-01, -3.1008e-01, -1.7403e-01,
         4.3029e-01,  1.6816e-01,  1.0590e-01, -4.8987e-01,  3.1856e-01,
         3.2861e-01, -1.3403e-02,  1.8807e-01, -1.0905e+00,  2.1009e-01,
        -6.7579e-01, -5.7076e-01,  8.5947e-02,  1.9121e-01, -3.3818e-01,
         2.7744e-01, -4.0539e-01,  3.1305e-01, -4.1197e-01, -5.6820e-01,
        -3.9074e-01,  4.0747e-01,  9.9898e-02,  2.3719e-01,  1.0154e-01,
        -2.5670e-01, -2.0583e-01,  1.1762e-01, -5.1439e-01,  4.0979e-01,
         1.2149e-01,  1.9333e-02, -5.9029e-02, -2.0141e-01,  7.0860e-01,
        -6.4609e-02,  2.4779e-02, -9.0578e-03,  1.9666e-02,  3.0815e-01,
        -4.9832e-02, -1.0691e+00,  6.1072e-01, -4.9722e-02, -1.5156e-01,
        -6.7778e-02,  4.7812e-02,  5.2103e-01,  1.6951e-01,  1.0146e-02,
```

```

5.3093e-01, -7.8189e-02, 6.5843e-02, -2.9382e-01, -4.6045e-01,
4.2071e-01, 1.1822e-01, 2.3631e-01, -4.5379e-02, -1.3740e-01,
-4.4018e-01, -6.8123e-02, 1.9935e-01, 8.7062e-01, -2.2603e-01,
3.3604e-01, 2.0236e-01, 3.7898e-01, 1.9533e-01, -3.0366e-01,
3.8633e-01, 6.1949e-01, 6.8663e-01, -1.8968e-01, -3.6815e-01,
-1.6616e-01, -7.0827e-02, -3.4610e-01, -8.5326e-01, 4.6645e-02,
2.8512e-01, 1.0890e-01, 2.5938e-01, -4.2975e-01, 4.3345e-01,
2.0637e-01, -3.8656e-01, -3.8187e-02, 3.6925e-01, 3.0130e-01,
4.0251e-01, 1.2887e-01, -3.7689e-01, -3.4447e-01, -4.2116e-01,
-1.0252e-01, -8.9737e-02, 4.7384e-01, 8.1717e-02, 1.5885e-01,
7.6674e-01, 3.4493e-01, 9.8538e-04, 4.8932e-02, 2.6132e-01,
3.8329e-02, -2.0036e-01, 2.6654e-01, 9.3773e-02, -4.6779e-02,
-4.0519e-01, -4.4310e-01, 6.1268e-01, -1.8950e-01, -3.8333e-01,
2.0583e-01, 1.5379e-01, -1.4664e-01, 5.3847e-01, -3.9618e-01,
-2.0599e+00, 6.7052e-01, 2.1112e-01, -4.7306e-01, 3.4865e-01,
-2.9919e-01, 5.4614e-01, -5.3924e-01, -2.4877e-01, -2.9070e-02,
-2.0319e-01, -7.3275e-02, -3.8147e-01, -5.4454e-01, 3.5049e-01,
-1.1249e-01, -2.1471e-01, -3.8439e-01, -1.0760e-01, -8.8821e-02,
2.5263e-01, 2.1448e-01, 5.5799e-02, -6.5411e-02, 9.9837e-02,
3.3435e-01, 2.4018e-01, 2.9875e-02, -1.1191e-01, 5.4330e-01,
-5.5214e-01, 1.1125e+00, 5.4141e-01, -7.4160e-02, 3.5337e-01,
1.2313e-01, 3.4855e-02, -2.8568e-01, -1.2517e-01, -4.4332e-02,
1.3323e-01, -2.4995e-01, -4.9833e-01, 4.1959e-01, -3.1580e-01,
6.1942e-01, 3.1113e-01, 4.8846e-01, 6.1518e-01, -3.6326e-02,
2.1294e-02, -3.5715e-01, 5.9126e-01, 1.5102e-01, -2.9641e-01,
2.9441e-01, -1.4138e-01, 1.1662e-01, -3.6223e-01, -1.4621e-01,
6.5254e-02, 3.9270e-01, 3.8543e-01, -2.3996e-01, -3.1482e-01,
-4.6860e-01, -1.1920e-01, 8.6236e-02, -3.4596e-02, -3.6275e-01,
-3.9838e-01, -3.6006e-01, -1.9672e-01, -2.7738e-01, -4.1097e-01,
3.6456e-01, -2.6012e-01, 1.2587e-01, 1.2752e-01, 5.4261e-01,
1.0569e-01, 3.5704e-01, 1.4766e-01, 4.4929e-01, -8.1255e-01,
-3.0409e-02, 5.8063e-02, 2.0699e-01, 6.6129e-01, 3.9243e-01,
-6.8644e-01, -8.3415e-01, -1.2653e-01, 1.9644e-01, -4.0900e-01,
-6.3777e-02, -1.8780e-01, 7.9473e-02, -1.7443e-01, 3.1936e-01,

```

- Vector biểu diễn câu có 768 chiều, tương ứng với kích thước tầng ẩn (hidden size) của BERT.
- attention_mask giúp Mean Pooling chỉ tính trung bình trên token thật, không bị nhiễu bởi padding.